

SQL

Presented By
Sylvia Anter



SQL



DQL

Select
Select' Family
1- from
2- where
3- Group by
4- having
5- Order by



DDL

1- create
2- alter
3- drop



DML

1- Insert
2- Update
3- Delete

Create , Alter , Drop

Create

Create Database
choose_name

create table
Choose_name



Al ter

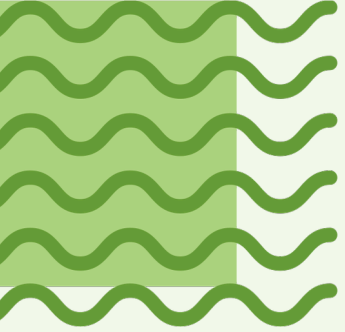
Alter table Ex_Sales
add column_name int



Drop

alter table Ex_Sales
Drop Column Column_name





Create



Create database Sales_Info

Create :



create table customers

```
(  
  CustomerKey int primary key ,  
  GeographyKey int ,  
  Customer varchar(30) Not null,  
  Continent varchar(10),  
  Region varchar(10)  
)
```

Create :



create table sales

```
(  
  SON int primary key,  
  productkey int,  
  CustomerKey int,  
  qty float not null,  
  unitprice float not null,  
  revenue float not null,  
  OrderDate date,
```

```
  foreign key (productkey) references products(productkey),  
  foreign key (CustomerKey) references customers(CustomerKey)  
)
```

Insert , Update , Delete

Insert

Insert into
Table_name
values()



Update

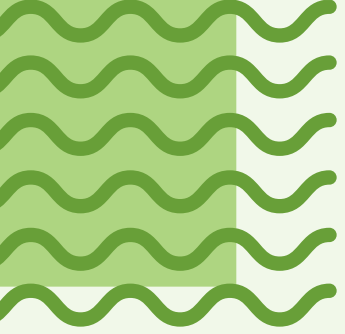
Update Table_name
Set column_name=new_value
where condition



Delete

Delete from Table_name
where Condition





Insert :



insert into customers
values
(111,110,'ahmed','africa','Egypt'),
(112,110,'ahmed','africa','soria'),
(113,110,'ahmed','africa','dubi')

Delete :



Delete from customers
where CustomerKey = 111

Update :



update customers
set Customer = 'olaa'
where CustomerKey = 112

customer_id	ref_id	name	region	country
111	110	ahmed	africa	Egypt
112	110	ahmed	africa	soria
113	110	ahmed	africa	soria
113	110	ahmed	africa	dubi

Select Family

Select

1- from

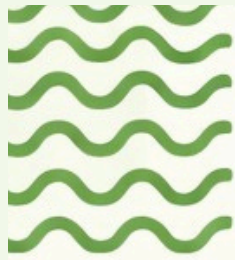
2- where

3- Group by

4- having

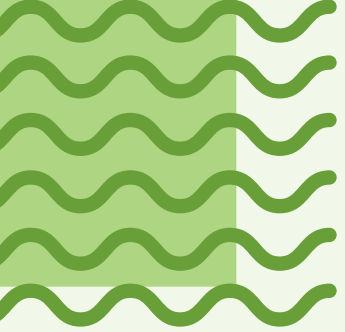
5- Order by





Sample Employee Table

id	first_name	last_name	salary	sex
10	Ahmed	Ali	4000	M
20	Sara	Mohamed	6000	F
30	Omar	Hassan	9000	M
40	Mona	Ibrahim	12000	F
50	Khaled	Youssef	7000	M
60	Nour	Adel	3000	F
70	Hany	Fathy	8500	M
Nulla	Laila	Samir	5000	F



Select :



Select *
from Employee

Select :



Select first_name , Last_Name
from Employee

Select :



Select first_name , Last_Name
from Employee
where salary > 5000

Select :



Select first_name , Last_Name
from Employee
where first_name = "Ahmed"

Select :

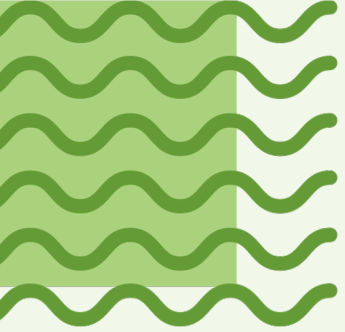


Select first_name , Last_Name
from Employee
where salary between 5000 and 9000

Select :



Select first_name , Last_Name
from Employee
where salary > 5000 and Sex = "M"



Select :



```
Select frist_name , Last_Name  
from Employee  
where salary not between 5000 and 9000
```

Select :



```
Select frist_name , Last_Name  
from Employee  
where salary > 5000 or Sex = "M"
```

Select :



```
Select frist_name , Last_Name  
from Employee  
where id in (10,20,30)
```

Select :



```
Select frist_name , Last_Name  
from Employee  
where id not in (10,20,30)
```

Select :

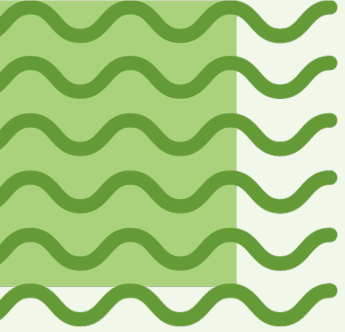


```
Select frist_name , Last_Name  
from Employee  
where id is Null
```

Select :



```
Select frist_name , Last_Name  
from Employee  
where id is not Null
```



Select :



```
Select frist_name , Last_Name  
from Employee  
where first_name like 'a%'
```

Select :



```
Select frist_name , Last_Name  
from Employee  
where first_name like '%d'
```

Select :



```
Select frist_name , Last_Name  
from Employee  
where first_name like '%a%'
```

Select :



```
Select frist_name , Last_Name  
from Employee  
where first_name like '_a%'
```

Select :

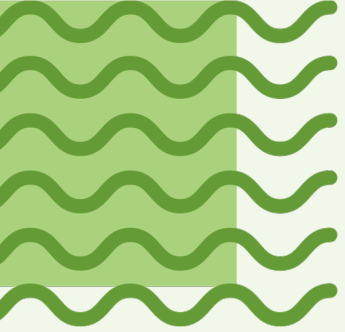


```
Select frist_name , Last_Name  
from Employee  
where first_name like '___a%'
```

Select :



```
Select frist_name , Last_Name  
from Employee  
Order by salary
```



Select :



```
Select frist_name , Last_Name  
from Employee  
where first_name like 'a%'
```

Select :



```
Select frist_name , Last_Name  
from Employee  
where first_name like '%d'
```

Select :



```
Select frist_name , Last_Name  
from Employee  
where first_name like '%a%'
```

Select :



```
Select frist_name , Last_Name  
from Employee  
where first_name like '_a%'
```

Select :

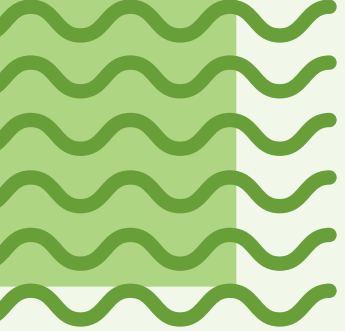


```
Select frist_name , Last_Name  
from Employee  
where first_name like '___a%'
```

Select :



```
Select frist_name , Last_Name  
from Employee  
Order by salary
```



Select :



Select min(Salary)
from employee

3000

Select :



Select max(Salary)
from employee

12000

Select :



Select sex , sum(Salary)
from employee
grouby by sex

SEX	Sum (Salary)
Male	10,000
Female	20,000



Sample Employee Table

id	first_name	last_name	salary	sex
10	Ahmed	Ali	4000	M
20	Sara	Mohamed	6000	F
30	Omar	Hassan	9000	M
40	Mona	Ibrahim	12000	F
50	Khaled	Youssef	7000	M
60	Nour	Adel	3000	F
70	Hany	Fathy	8500	M
Nulla	Laila	Samir	5000	F

Types Of Join



Inner Join



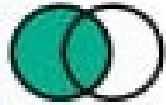
Left Join & Right Join



Full Join

Join Types

Left Outer Join



Key	T1
B	#
C	@
D	=
E	\$

Left Table

Key	T1	T2
B	#	15
C	@	20
D	=	
E	\$	

Key	T2
A	10
B	15
C	20
F	60

Right Table

All rows from Left + **matching** rows from Right

Right Outer Join



Key	T1
B	#
C	@
D	=
E	\$

Left Table

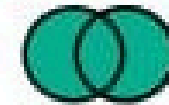
Key	T1	T2
A		10
B	#	15
C	@	20
F		60

Key	T2
A	10
B	15
C	20
F	60

Right Table

All rows from Right + **matching** rows from Left

Full Outer Join



Key	T1
B	#
C	@
D	=
E	\$

Left Table

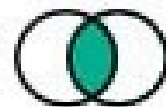
Key	T1	T2
A		10
B	#	15
C	@	20
D	=	
E	\$	
F		60

Key	T2
A	10
B	15
C	20
F	60

Right Table

All rows from both

Inner Join



Key	T1
B	#
C	@
D	=
E	\$

Left Table

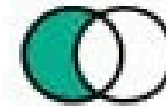
Key	T1	T2
B	#	15
C	@	20

Key	T2
A	10
B	15
C	20
F	60

Right Table

Matching rows between Left and Right

Left Anti Join



Key	T1
B	#
C	@
D	=
E	\$

Left Table

Key	T1	T2
D	=	
E	\$	

Key	T2
A	10
B	15
C	20
F	60

Right Table

Rows from Left **not matching** with Right

Right Anti Join



Key	T1
B	#
C	@
D	=
E	\$

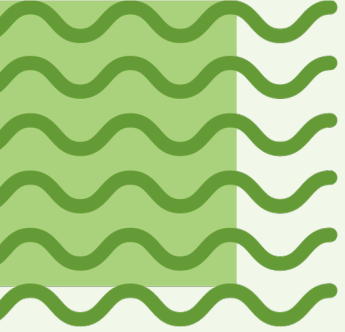
Left Table

Key	T1	T2
A		10
F		60

Key	T2
A	10
B	15
C	20
F	60

Right Table

Rows from Right **not matching** with Left



Inner Join :



```
select *  
from customers join sales  
on customers.CustomerKey = sales.customerkey
```

left join :



```
select *  
from customers left join sales  
on customers.CustomerKey = sales.customerkey
```

Right Join :

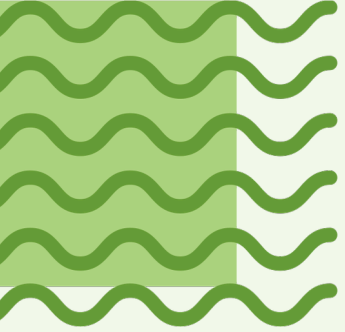


```
select *  
from customers right join sales  
on customers.CustomerKey = sales.customerkey
```

Full Join:



```
select *  
from customers full join sales  
on customers.CustomerKey = sales.customerkey
```



Select :



```
select Size , sum(revenue) as "total_Revenue"  
from products inner join sales  
on products.ProductKey=sales.productkey  
group by Size  
having sum(revenue) > 15000
```

Size	Sum (Revenue)
M	10,000
L	20,000

Select :



```
select Continent , sum(revenue) as "total_Revenue"  
from customers inner join sales  
on customers.CustomerKey=sales.customerkey  
group by Continent  
having sum(revenue) > 150,000
```

Continent	Sum (Revenue)
Africa	100,000
Eurpoe	250,000

SubQuer y



```
SELECT column_name  
FROM table_name  
WHERE column_name = (  
    SELECT column_name  
    FROM another_table  
)
```

DeptID	Salary	Name	EmplD
1	8000	Ahmed	1
2	12000	Sara	2
1	10000	Mona	3

```
SELECT Name, Salary  
FROM Employees  
WHERE Salary >  
    (  
    SELECT AVG(Salary)  
    FROM Employees  
    )
```

عايزين نجيب الموظفين اللي مرتبتهم أعلى من
متوسط المرتبات

get the employees that their salaries
greater than the average of salaries

DeptID	Salary	Name	EmplID
1	8000	Ahmed	1
2	12000	Sara	2
1	10000	Mona	3

```
SELECT Name
FROM Employees
WHERE DeptID IN (
    SELECT DeptID
FROM Departments
WHERE ManagerID IS NOT NULL
)
```

عايزين الموظفين اللي شغالين في الأقسام اللي
فيها مديرين

get the employees that their department
has manager .



How I Can Remove space from text ?

select your name ' sylvia '

then select your name without space

select ' sylvia '

select trim(' sylvia ')



How Can I make all the text upper or lower ?

select UPPER('sylvia')

select UPPER('sylvIA')

select UPPER('Sylvia')



How I Can get the length of text ?

```
select len('3020601218096')  
select len('IN-120')
```



How Can I replace value with another ?

```
select sex , REPLACE(sex,'F','female')  
from Employee
```

```
SELECT sex, REPLACE(REPLACE(sex, 'F', 'Girl'), 'M', 'Boy' ) AS New_Sex  
FROM Employee;
```

```
UPDATE Employee  
SET sex = REPLACE(REPLACE(sex, 'F', 'Girls'), 'M', 'Boy');
```



How I Can extract sub string from text ?

```
select first_name , left(first_name,3)
      from Employee
```

```
select first_name , right(first_name,3)
      from Employee
```

```
select first_name , substring(first_name,1,3)
      from Employee
```



How I Can get the index of one char as python ?

```
select first_name , CHARINDEX('A',first_name)
      from Employee
```



How I Can extract user name from email column ?

```
select 'sylviaanter@gmail.com' , CHARINDEX('@','sylviaanter@gmail.com')
```

```
select 'sylviaanter@gmail.com'  
,left('sylviaanter@gmail.com',CHARINDEX('@','sylviaanter@gmail.com')-1)
```



Tracing

```
select 'ln-120' , CHARINDEX('-', 'ln-120')
```

```
select 'ln-120' , Right('ln-120', CHARINDEX('-', 'ln-120') + 1)
```



Casting in Sql ?

```
select sex , cast(sum(salary) as int)
      from employee
      group by sex
```



float Point

```
select sex , round(sum(salary),0)
      from employee
      group by sex
```

Case

SalaryCategory	Salary	Name
Low Salary	8000	Ahmed
High Salary	12000	Sara
High Salary	10000	Mona

```
select first_name , salary ,  
       Case  
when salary > 8000 then 'High Salary'  
when salary < 5000 then 'Low Salary'  
       Else 'Normal'  
end as 'Cattategory'  
from employee
```

عائزين نصف المرتبات

Salary	Sex	Department	Name	id
4000	M	HR	Ahmed	10
6000	F	HR	Sara	20
9000	M	IT	Omar	30
12000	F	IT	Mona	40
7000	M	Sales	Khaled	50
3000	F	Sales	Nour	60
8500	M	Finance	Hany	70
5000	F	Finance	Laila	80

window function

SQL window functions perform calculations across a set of table rows that are related to the current row, but unlike aggregate functions

Window Function

Group by

	department	(No column name)
1	Finance	13500
2	HR	10500
3	IT	21000
4	Sales	10000

	id	first_name	department	sex	salary	Total_Department
1	70	Hany	Finance	M	8500	13500
2	80	Laila	Finance	F	5000	13500
3	10	Ahmed	HR	M	4000	10500
4	20	Sara	HR	F	6500	10500
5	30	Omar	IT	M	9000	21000
6	40	Mona	IT	F	12000	21000
7	50	Khaled	Sales	M	7000	10000
8	60	Nour	Sales	F	3000	10000

step 1 add new column – Department

```
ALTER TABLE Employee  
ADD department VARCHAR(20);
```

```
UPDATE Employee  
SET department =  
    CASE  
        WHEN id = 10 THEN 'HR'  
        WHEN id = 20 THEN 'HR'  
        WHEN id = 30 THEN 'IT'  
        WHEN id = 40 THEN 'IT'  
        WHEN id = 50 THEN 'Sales'  
        WHEN id = 60 THEN 'Sales'  
        WHEN id = 70 THEN 'Finance'  
        WHEN id = 80 THEN 'Finance'  
    END;
```

Salary	Sex	Department	Name	id
4000	M	HR	Ahmed	10
6000	F	HR	Sara	20
9000	M	IT	Omar	30
12000	F	IT	Mona	40
7000	M	Sales	Khaled	50
3000	F	Sales	Nour	60
8500	M	Finance	Hany	70
5000	F	Finance	Laila	80

window function

Group by

	department	(No column name)
1	Finance	13500
2	HR	10500
3	IT	21000
4	Sales	10000

```
SELECT
    Department,
    SUM(salary) AS Total_Salary
FROM Employee
GROUP BY Department;
```

Window Function

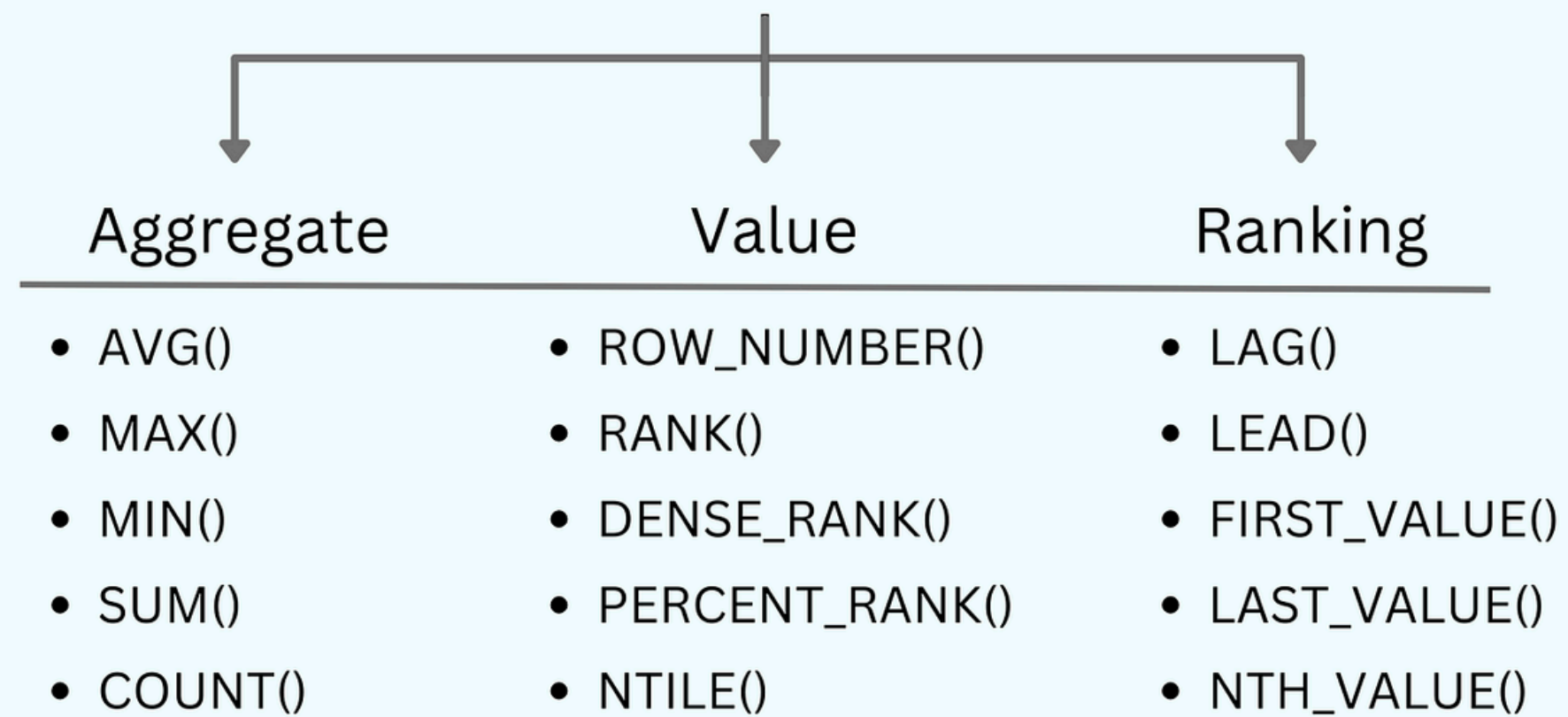
	id	first_name	department	sex	salary	Total_Department
1	70	Hany	Finance	M	8500	13500
2	80	Laila	Finance	F	5000	13500
3	10	Ahmed	HR	M	4000	10500
4	20	Sara	HR	F	6500	10500
5	30	Omar	IT	M	9000	21000
6	40	Mona	IT	F	12000	21000
7	50	Khaled	Sales	M	7000	10000
8	60	Nour	Sales	F	3000	10000

```
SELECT id,first_name,department, sex, salary,
    SUM(salary) OVER(PARTITION BY department) AS
        Total_Department
FROM Employee;
```

window function



Window Functions



Window Function

1- Over() - Sum()

	id	first_name	department	sex	salary	Total_Department
1	70	Hany	Finance	M	8500	13500
2	80	Laila	Finance	F	5000	13500
3	10	Ahmed	HR	M	4000	10500
4	20	Sara	HR	F	6500	10500
5	30	Omar	IT	M	9000	21000
6	40	Mona	IT	F	12000	21000
7	50	Khaled	Sales	M	7000	10000
8	60	Nour	Sales	F	3000	10000

```
SELECT id,first_name,department, sex, salary,  
SUM(salary) OVER(PARTITION BY department) AS  
Total_Department  
FROM Employee;
```

Window Function

2- Over() - Avg()

	id	first_name	department	sex	salary	AVG_Salary
1	70	Hany	Finance	M	8500	6750
2	80	Laila	Finance	F	5000	6750
3	10	Ahmed	HR	M	4000	5250
4	20	Sara	HR	F	6500	5250
5	30	Omar	IT	M	9000	10500
6	40	Mona	IT	F	12000	10500
7	50	Khaled	Sales	M	7000	5000
8	60	Nour	Sales	F	3000	5000

```
SELECT id,first_name,department, sex, salary,  
AVG(salary) OVER(PARTITION BY department) AS  
AvgSalary  
FROM Employee;
```

Window Function

3- Over() - Count()

	first_name	department	Emp_Count
1	Hany	Finance	2
2	Laila	Finance	2
3	Ahmed	HR	2
4	Sara	HR	2
5	Omar	IT	2
6	Mona	IT	2
7	Khaled	Sales	2
8	Nour	Sales	2

```
SELECT first_name,department  
count(*) OVER(PARTITION BY department) AS  
Emp_Count  
FROM Employee;
```

Window Function

4- Over() - Row_Number()

first_name	salary	RowNum
Mona	12000	1
Omar	9000	2
Hany	8500	3
Khaled	7000	4
Sara	6500	5
Laila	5000	6
Ahmed	4000	7
Nour	3000	8

```
SELECT first_name,department  
Row_Number() OVER(Order BY salary DESC) AS  
ROWNUM  
FROM Employee;
```

Window Function

5- Over() - Rank()

first_name	salary	RowNum
Mona	12000	1
Omar	9000	2
Hany	8500	3
Khaled	7000	4
Sara	6500	5
Laila	5000	6
Ahmed	4000	7
Nour	3000	8

```
SELECT first_name,department  
RANK() OVER(ORDER BY salary DESC) AS Rank_EMP  
FROM Employee;
```

Window Function

6- Over() - Dense Rank()

first_name	salary	RowNum
Mona	12000	1
Omar	9000	2
Hany	8500	3
Khaled	7000	4
Sara	6500	5
Laila	5000	6
Ahmed	4000	7
Nour	3000	8

```
SELECT first_name,department  
DENSE_RANK() OVER(ORDER BY salary DESC) AS Rank_EMP  
FROM Employee;
```

Window Function

7- Over - MIN() - Max()

	first_name	salary	MinSalary	MaxSalary
	Hany	8500	5000	8500
	Laila	5000	5000	8500
	Ahmed	4000	4000	6500
	Sara	6500	4000	6500
	Omar	9000	9000	12000
	Mona	12000	9000	12000
	Khaled	7000	3000	7000
	Nour	3000	3000	7000

```
SELECT first_name, salary,  
MIN(salary) OVER(PARTITION BY department) AS MinSalary,  
MAX(salary) OVER(PARTITION BY department) AS MaxSalary  
FROM Employee;
```

Window Function

8- Over() -Ntile()

first_name	salary	Group_No
Mona	12000	1
Omar	9000	1
Hany	8500	1
Khaled	7000	1
Sara	6500	2
Laila	5000	2
Ahmed	4000	2
Nour	3000	2

```
SELECT first_name, salary,  
NTILE(2) OVER(ORDER BY salary DESC) AS Group_No  
FROM Employee;
```

Window Function

9- Over() - LAG()

	first_name	salary	PreviousSalary
1	Nour	3000	NULL
2	Ahmed	4000	3000
3	Laila	5000	4000
4	Sara	6500	5000
5	Khaled	7000	6500
6	Hany	8500	7000
7	Omar	9000	8500
8	Mona	12000	9000

```
SELECT first_name, salary,  
LAG(salary) OVER(ORDER BY salary) AS PreviousSalary  
FROM Employee;
```

Window Function

9- Over() - Lead()

	first_name	salary	PreviousSalary
1	Nour	3000	NULL
2	Ahmed	4000	3000
3	Laila	5000	4000
4	Sara	6500	5000
5	Khaled	7000	6500
6	Hany	8500	7000
7	Omar	9000	8500
8	Mona	12000	9000

```
SELECT first_name, salary,  
Lead(salary) OVER(ORDER BY salary) AS next  
FROM Employee;
```

Window Function

Get the Running total

first_name	salary	Running_Total
Nour	3000	3000
Ahmed	4000	7000
Laila	5000	12000
Sara	6500	18500
Khaled	7000	25500
Hany	8500	34000
Omar	9000	43000
Mona	12000	55000

```
SELECT first_name, salary,  
       SUM(salary) OVER(  
         ORDER BY salary  
       ) AS Running_Total  
FROM Employee;
```

Window Function

Tracing

```
SELECT  
    first_name,  
    department,  
    salary,  
    ROW_NUMBER() OVER(  
        PARTITION BY department  
        ORDER BY salary DESC  
    ) AS Rank_In_Department  
FROM Employee;
```

CTE– Common Table Expression

A Common Table Expression (CTE) is a temporary, named result set in SQL . It acts like a virtual table that exists only during the execution of that specific query

```
with █ as ( █ █ █ )
```

```
select *
```

```
from █ where █ █ █
```

CTE

using CTE give me the employees that thier salaires greater than 6k

```
WITH HighSalary AS
(
    SELECT *
    FROM Employee
    WHERE Salary > 6000
)
SELECT *
FROM HighSalary;
```

id	first_name	last_name	salary	sex	department
20	Sara	Mohamed	6500	Girls	HR
30	Omar	Hassan	9000	Boy	IT
40	Mona	Ibrahim	12000	Girls	IT
50	Khaled	Youssef	7000	Boy	Sales
70	Hany	Fathy	8500	Boy	Finance

CTE

using CTE give me the employee name, dep , salary from IT department

```
WITH ITEmployees AS  
(  
    SELECT *  
    FROM Employee  
    WHERE Department = 'IT'  
)
```

```
SELECT first_name, Department, Salary  
FROM ITEmployees;
```

first_name	Department	Salary
Omar	IT	9000
Mona	IT	12000

CTE-order by

using CTE give me the employee that thier salaries > 5k and ordered DESC

```
WITH Employees AS
(
    SELECT *
    FROM Employee
    WHERE Salary > 5000
)
SELECT first_name, Department, Salary
FROM Employees
ORDER BY Salary DESC;
```

first_name	Department	Salary
Mona	IT	12000
Omar	IT	9000
Hany	Finance	8500
Khaled	Sales	7000
Sara	HR	6500

CTE-group by

using CTE calculate the avr of salaies for each dep

```
WITH DepartmentSalary AS
(
    SELECT
    Department,AVG(Salary) AS AvgSalary
    FROM Employee
    GROUP BY Department
)
SELECT *
FROM DepartmentSalary
```

Department	AvgSalary
Finance	6750
HR	5250
IT	10500
Sales	5000

CTE-group by

using CTE show the dep that thier avr of salary greater than 6000

```
WITH DepartmentSalary AS  
(  
    SELECT  
        Department,  
        AVG(Salary) AS AvgSalary  
    FROM Employee  
    GROUP BY Department  
)  
SELECT *  
FROM DepartmentSalary  
WHERE AvgSalary > 6000;
```

Department	AvgSalary
Finance	6750
HR	5250
IT	10500
Sales	5000

CTE-group by

using CTE get the max salary in each dep

```
WITH DepartmentMaxSalary AS  
(  
  SELECT Department, MAX(Salary) AS MaxSalary  
  FROM Employee  
  GROUP BY Department  
)  
SELECT *  
FROM DepartmentMaxSalary;
```

Department	MaxSalary
Finance	8500
HR	6500
IT	12000
Sales	7000

Department	MaxSalary
Finance	8500
HR	6500
IT	12000
Sales	7000

id	first_name	last_name	salary	sex	department
10	Ahmed	Ali	4000	Boy	HR
20	Sara	Mohamed	6500	Girls	HR
30	Omar	Hassan	9000	Boy	IT
40	Mona	Ibrahim	12000	Girls	IT
50	Khaled	Youssef	7000	Boy	Sales
60	Nour	Adel	3000	Girls	Sales
70	Hany	Fathy	8500	Boy	Finance
80	Laila	Samir	5000	Girls	Finance

CTE-join

using CTE get the max salary in each dep

```
WITH DepartmentMaxSalary AS
(
    SELECT Department, MAX(Salary) AS MaxSalary
    FROM Employee
    GROUP BY Department
)
SELECT e.first_name, e.Department, e.Salary
FROM Employee e inner JOIN DepartmentMaxSalary d
    ON e.Department = d.Department
    AND e.Salary = d.MaxSalary;
```

first_name	Department	Salary
Khaled	Sales	7000
Mona	IT	12000
Sara	HR	6500
Hany	Finance	8500

CTE-join

using CTE get each employee and the avr of salaries in thier dep

```
WITH DepartmentAvg AS
(
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY Department
)
SELECT e.first_name, e.Department , e.Salary, d.AvgSalary
FROM Employee e JOIN DepartmentAvg d
ON e.Department = d.Department;
```

first_name	Department	Salary	AvgSalary
Hany	Finance	8500	6750
Laila	Finance	5000	6750
Ahmed	HR	4000	5250
Sara	HR	6500	5250
Omar	IT	9000	10500
Mona	IT	12000	10500
Khaled	Sales	7000	5000
Nour	Sales	3000	5000

Multiple CTE

Tracing

```
WITH DepartmentAvg AS
(
SELECT Department,AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY Department
),
HighPayingDepartments AS
(
SELECT Department,AvgSalary
FROM DepartmentAvg
WHERE AvgSalary > 6000
)
SELECT e.first_name,e.Department,e.Salary
FROM Employee e JOIN HighPayingDepartments h
ON e.Department = h.Department;
```

first_name	Department	Salary
Hany	Finance	8500
Laila	Finance	5000
Omar	IT	9000
Mona	IT	12000

CTE- Window Function

using CTE Rank the employee according to the salary

```
WITH EmployeeRanking AS
(
    SELECT first_name, Department, Salary,
    RANK() OVER(ORDER BY Salary DESC) AS SalaryRank
    FROM Employee
)
SELECT *
FROM EmployeeRanking;
```

first_name	Department	Salary	SalaryRank
Mona	IT	12000	1
Omar	IT	9000	2
Hany	Finance	8500	3
Khaled	Sales	7000	4
Sara	HR	6500	5
Laila	Finance	5000	6
Ahmed	HR	4000	7
Nour	Sales	3000	8

CTE- Window Function

using CTE Rank the employee according to the salary in each dep

WITH EmployeeRanking AS

(

SELECT first_name,Department,Salary,

ROW_NUMBER() OVER(PARTITION BY Department

ORDER BY Salary DESC) AS RowNum

FROM Employee

)

SELECT

first_name,Department,Salary

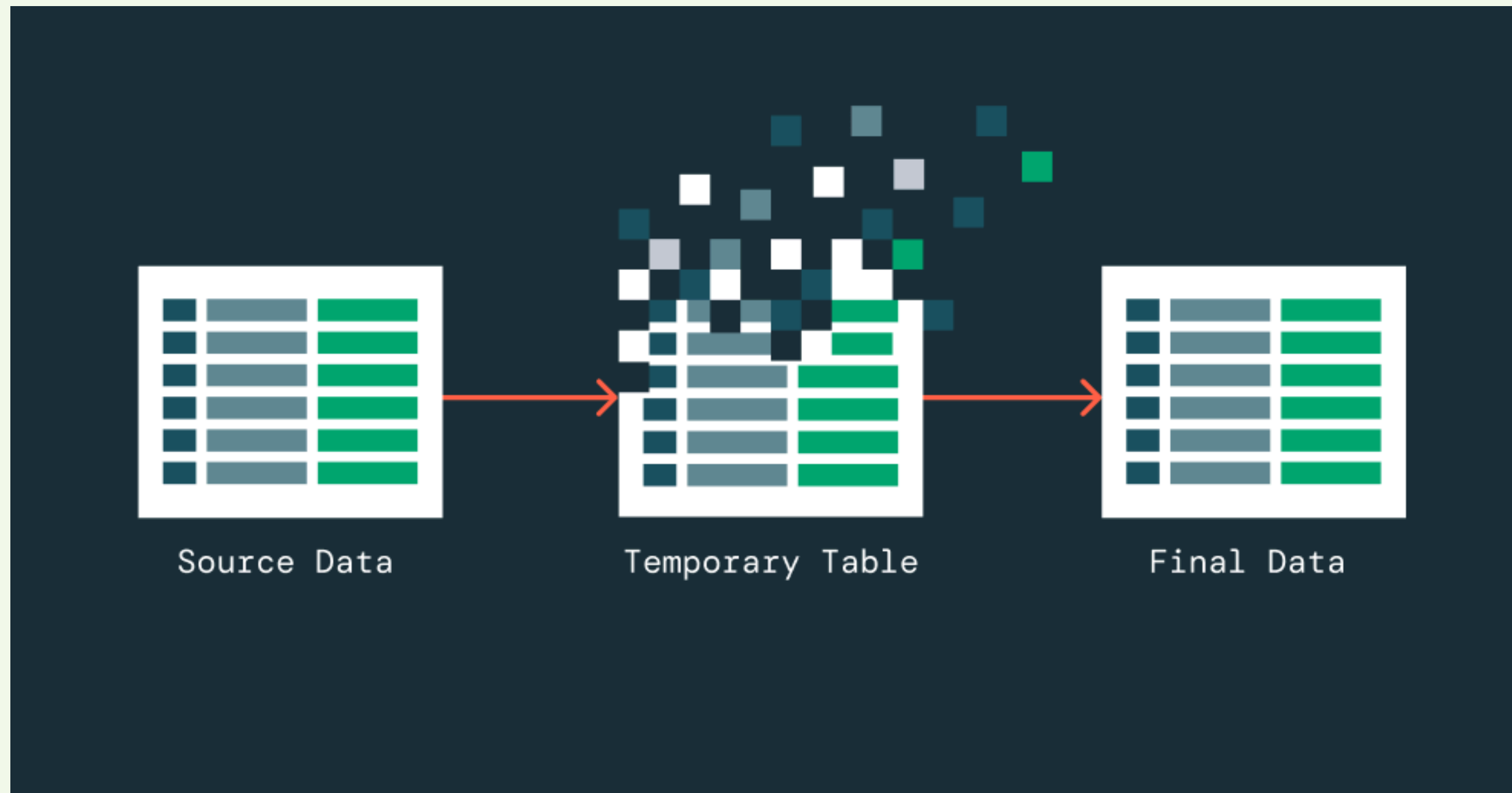
FROM EmployeeRanking

WHERE RowNum = 1;

first_name	Department	Salary
Hany	Finance	8500
Sara	HR	6500
Mona	IT	12000
Khaled	Sales	7000

Temp Table

a special table used to store data during a specific database session



Temp Table

- Create Temp Table for High Salary Employees

```
SELECT *  
INTO #HighSalary  
FROM Employee  
WHERE Salary > 6000;
```

```
SELECT *  
FROM #HighSalary;
```

- Create Temp Table for IT Employees

```
SELECT *  
INTO #ITEmployees  
FROM Employee  
WHERE Department = 'IT';
```

- Create a temporary table containing the average salary for each department.

```
SELECT Department, AVG(Salary) AS AvgSalary  
INTO #DepartmentSalary  
FROM Employee  
GROUP BY Department;
```

Temp Table

- Find departments where the average salary is greater than 6000.

```
SELECT *  
FROM #DepartmentSalary  
WHERE AvgSalary > 6000;
```

- Show each employee with the average salary of their department.

```
SELECT e.Name, e.Department, e.Salary, d.AvgSalary  
FROM Employee e JOIN #DepartmentSalary d  
ON e.Department = d.Department;
```

Temp Table

- Create a temporary table containing all employees, then increase their salaries by 10%.

```
SELECT *  
INTO #EmployeeTemp  
FROM Employee;
```

```
UPDATE #EmployeeTemp  
SET Salary = Salary * 1.10;
```

```
SELECT *  
FROM #EmployeeTemp;
```

- Create a temporary table containing all employees, then delete employees whose salary is less than 5000.

```
SELECT *  
INTO #EmployeeTemp  
FROM Employee;
```

```
DELETE FROM #EmployeeTemp  
WHERE Salary < 5000;
```

```
SELECT *  
FROM #EmployeeTemp;
```

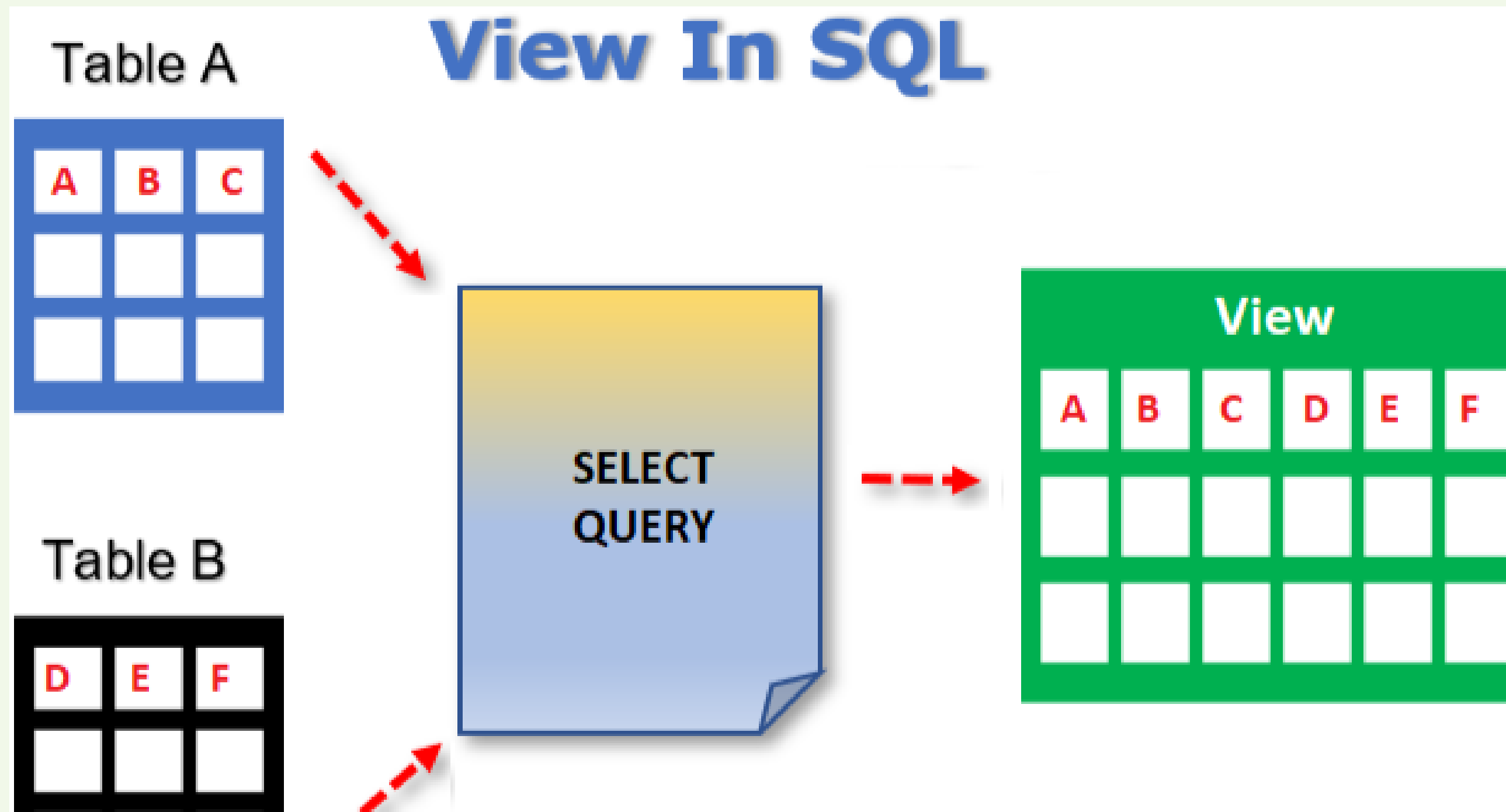
				Department
ManagerName	Location	DepartmentName	DepartmentID	
Ahmed	Cairo	HR	1	
Mona	Alexandria	IT	2	
Khaled	Giza	Sales	3	
Hany	Cairo	Finance	4	

```
CREATE TABLE Department
(
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50),
    Location VARCHAR(50),
    ManagerName VARCHAR(50)
);
```

```
INSERT INTO Department
VALUES
(1, 'HR', 'Cairo', 'Ahmed'),
(2, 'IT', 'Alexandria', 'Mona'),
(3, 'Sales', 'Giza', 'Khaled'),
(4, 'Finance', 'Cairo', 'Hany');
```

View

An SQL view is a table based on the result-set of an SQL statement.



View

Create a view that shows employee name, department, salary, and department location.

```
CREATE VIEW EmployeeDepartmentInfo
AS
```

```
SELECT e.first_name,e.Department,e.Salary,d.Location
FROM Employee e JOIN Department d
ON e.Department = d.DepartmentName;
```

```
SELECT *
FROM EmployeeDepartmentInfo;
```

first_name	Department	Salary	Location
Ahmed	HR	4000	Cairo
Sara	HR	6500	Cairo
Omar	IT	9000	Alexandria
Mona	IT	12000	Alexandria
Khaled	Sales	7000	Giza
Nour	Sales	3000	Giza
Hany	Finance	8500	Cairo
Laila	Finance	5000	Cairo

View

Create a view showing employee name, department, salary, and department manager.

CREATE VIEW EmployeeManagerInfo

AS

SELECT *

SELECT e.first_name,e.Department,e.Salary,d.ManagerName **FROM** EmployeeManagerInfo;

FROM Employee e **JOIN** Department d

ON e.Department = d.DepartmentName;

EmployeeName	Department	Salary	ManagerName
Ahmed	HR	4000	Ahmed
Sara	HR	6500	Ahmed
Omar	IT	9000	Mona
Mona	IT	12000	Mona
Khaled	Sales	7000	Khaled
Nour	Sales	3000	Khaled
Hany	Finance	8500	Hany
Laila	Finance	5000	Hany

View

Create a view that shows employees with salary greater than 6000, along with their department location.

```
CREATE VIEW HighSalaryEmployeeInfo
```

```
AS
```

```
SELECT *
```

```
SELECT e.first_name, e.Department,e.Salary,d.Location FROM HighSalaryEmployeeInfo;
```

```
FROM Employee e JOIN Department d
```

```
ON e.Department = d.DepartmentName
```

```
WHERE e.Salary > 6000;
```

first_name	Department	Salary	Location
Sara	HR	6500	Cairo
Omar	IT	9000	Alexandria
Mona	IT	12000	Alexandria
Khaled	Sales	7000	Giza
Hany	Finance	8500	Cairo

View

Create a view showing the number of employees and average salary for each department.

```
CREATE VIEW DepartmentStatistics
AS
SELECT d.DepartmentName, d.Location,
COUNT(e.id) AS EmployeeCount,
AVG(e.Salary) AS AvgSalary
FROM Department d LEFT JOIN Employee e
ON d.DepartmentName = e.Department
GROUP BY
d.DepartmentName,
d.Location;
```

DepartmentName	Location	EmployeeCount	AvgSalary
IT	Alexandria	2	10500
Finance	Cairo	2	6750
HR	Cairo	2	5250
Sales	Giza	2	5000

```
SELECT *
FROM DepartmentStatistics;
```

View

Show departments where average salary is greater than 6000.from prev.Table

```
SELECT *  
FROM DepartmentStatistics  
WHERE AvgSalary > 6000;
```

DepartmentName	Location	EmployeeCount	AvgSalary
IT	Alexandria	2	10500
Finance	Cairo	2	6750

View

Create a view that ranks employees by salary within each department.

```
CREATE VIEW EmployeeDepartmentRanking AS
```

```
SELECT
```

```
    e.id,
```

```
    e.first_name,
```

```
    e.Department,
```

```
    e.Salary,
```

```
ROW_NUMBER() OVER
```

```
    (
```

```
    PARTITION BY e.Department
```

```
    ORDER BY e.Salary DESC
```

```
    ) AS SalaryRank
```

```
FROM Employee e;
```

id	first_name	Department	Salary	SalaryRank
70	Hany	Finance	8500	1
20	Sara	HR	6500	1
40	Mona	IT	12000	1
50	Khaled	Sales	7000	1

```
SELECT *
```

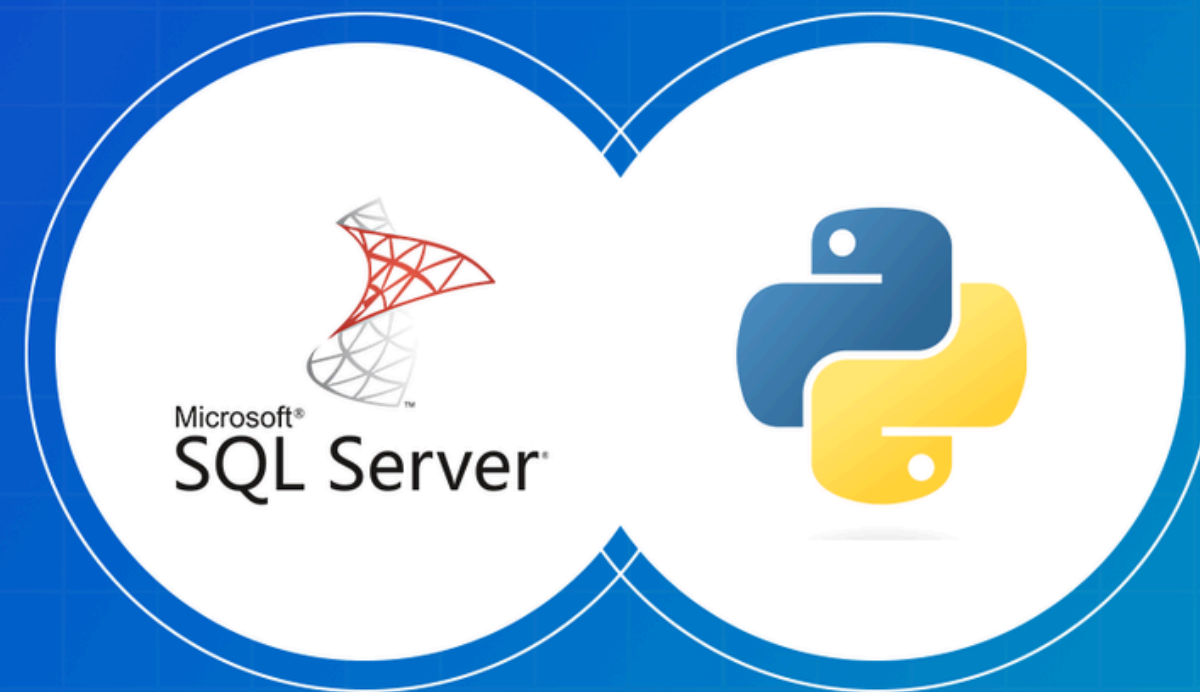
```
FROM
```

```
EmployeeDepartmentRanking
```

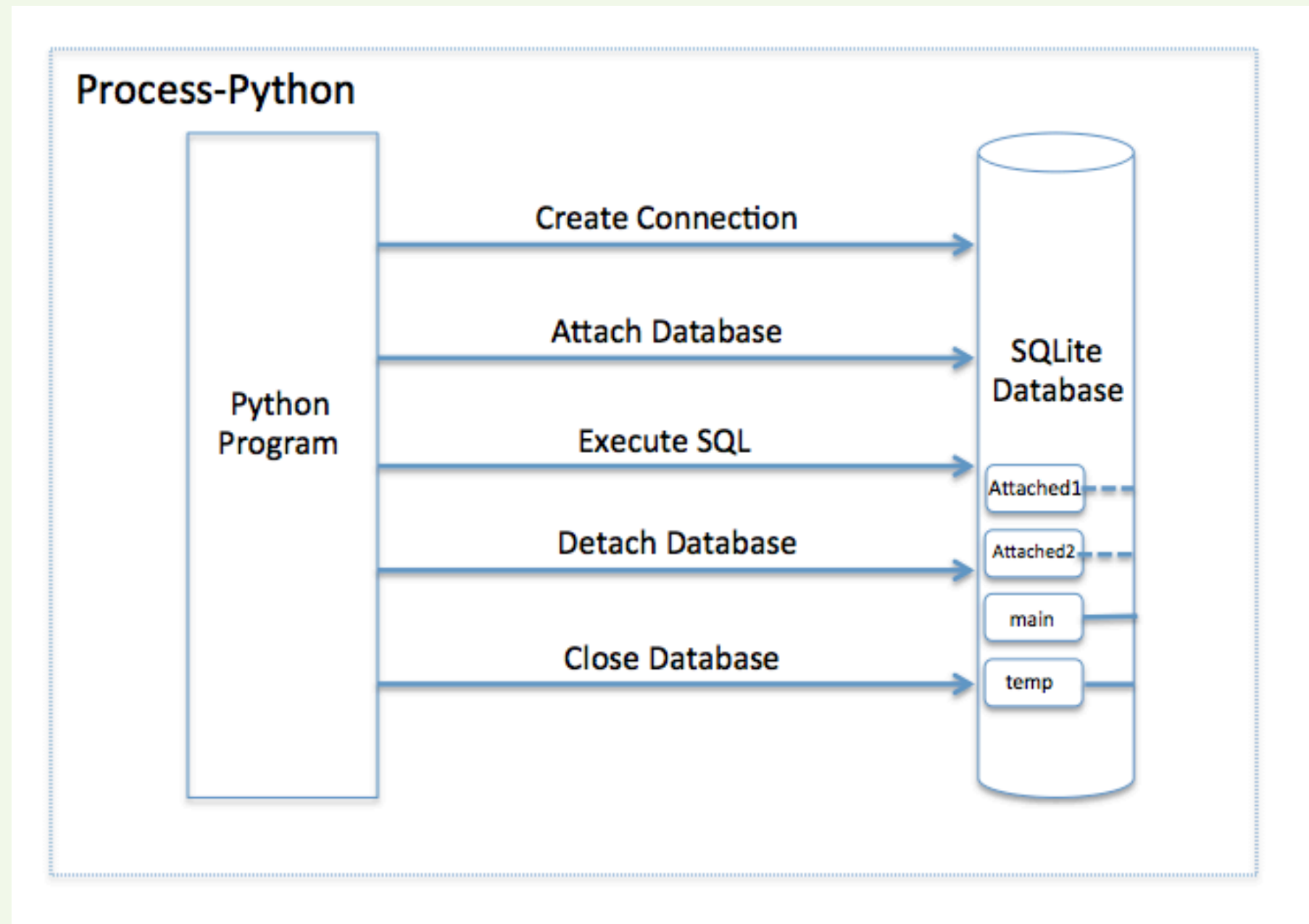
```
WHERE SalaryRank = 1;
```

SQL & Python

SQL Server Integration With Python

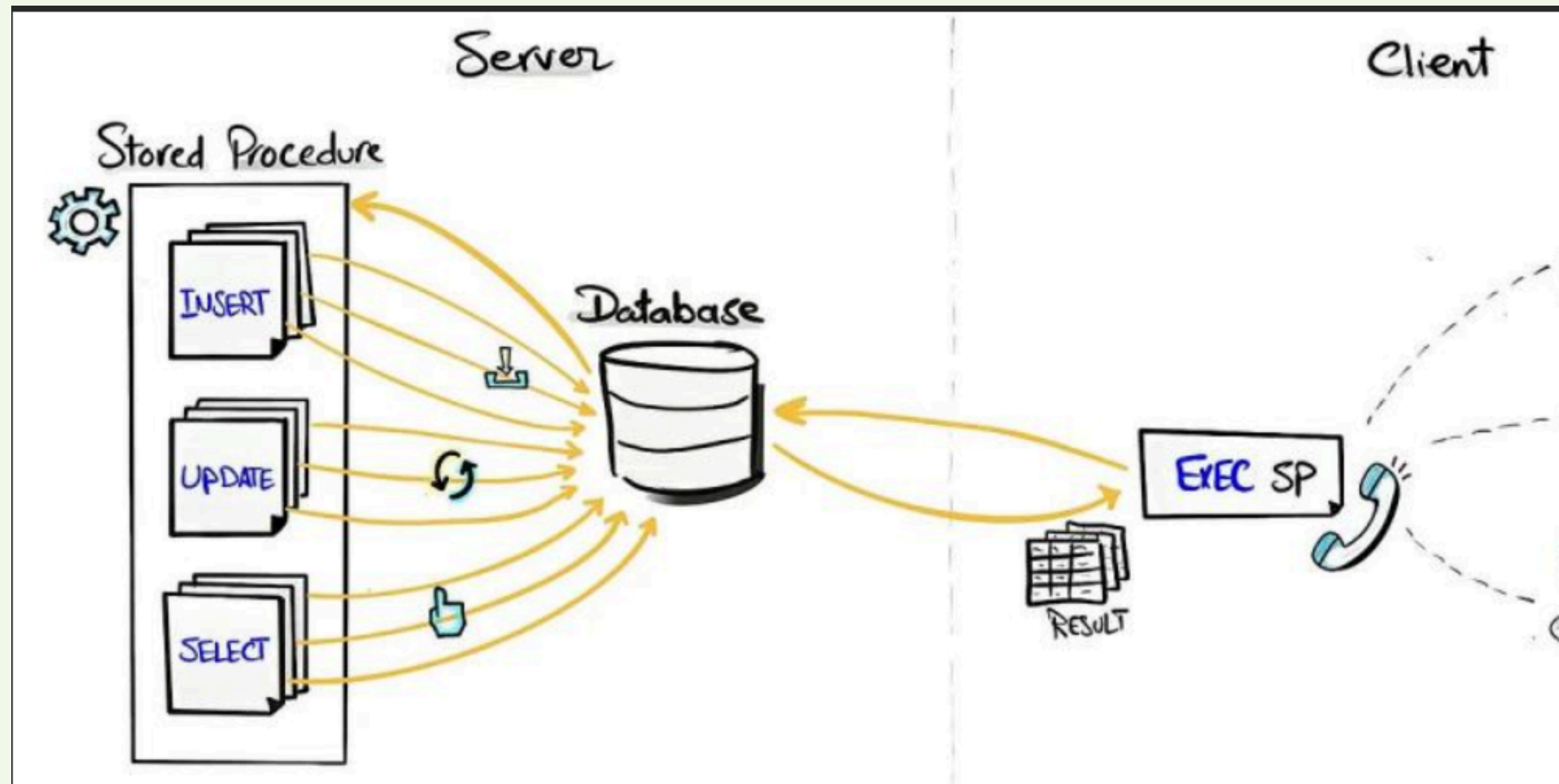


```
import sqlite3
import pandas as
pd
I want to create Database ?
db=sqlite3.connect('database_file.db')
db.close()
```



Stored Procedure

a saved set of SQL code stored directly inside a database



get the info of specific student

```
CREATE PROCEDURE GetStudentInfo  
    @StudentID INT  
    AS  
    BEGIN
```

```
    SELECT s.StudentID, s.FirstName, s.LastName, s.Gender, s.BirthDate, s.Email, d.DepartmentName, s.GPA  
    FROM Students s JOIN Departments d  
    ON s.DepartmentID = d.DepartmentID  
    WHERE s.StudentID = @StudentID;  
    END;
```

```
EXEC GetStudentInfo @StudentID = 1;
```

StudentID	FirstName	LastName	Gender	BirthDate	Email	DepartmentName	GPA
1	Ahmed	Ali	Male	2002-05-10	ahmed.ali@email.com	Computer Science	3.5

get the students in a specific department

```
CREATE PROCEDURE GetStudentsByDepartment
    @DepartmentName VARCHAR(100)
    AS
    BEGIN
        SELECT s.StudentID, s.FirstName, s.LastName, s.GPA, d.DepartmentName
            FROM Students s JOIN Departments d
            ON s.DepartmentID = d.DepartmentID
        WHERE d.DepartmentName = @DepartmentName
        ORDER BY s.GPA DESC;
    END;
```

```
EXEC GetStudentsByDepartment
@DepartmentName = 'Computer Science';
```

StudentID	FirstName	LastName	GPA	DepartmentName
1	Ahmed	Ali	3.5	Computer Science
7	Karim	Mostafa	3.1	Computer Science
3	Omar	Hassan	2.9	Computer Science

get the students in a specific course

```
CREATE PROCEDURE GetCourseStudents
    @CourseID INT
AS
BEGIN
    SELECT c.CourseName, s.StudentID, s.FirstName, s.LastName, e.EnrollmentDate, e.Grade
        FROM Enrollments e JOIN Students s
            ON e.StudentID = s.StudentID
            JOIN Courses c
            ON e.CourseID = c.CourseID
        WHERE c.CourseID = @CourseID
        ORDER BY e.Grade DESC;
END;

EXEC GetCourseStudents
    @CourseID = 101;
```

CourseName	StudentID	FirstName	LastName	EnrollmentDate	Grade
SQL Fundamentals	6	Nour	Samir	2026-01-17	98
SQL Fundamentals	2	Sara	Mohamed	2026-01-15	95
SQL Fundamentals	1	Ahmed	Ali	2026-01-15	88

get the avr of student grades

```
CREATE PROCEDURE GetStudentAverageGrade
    @StudentID INT
AS
BEGIN
    SELECT s.StudentID, s.FirstName, s.LastName, AVG(e.Grade) AS AverageGrade
    FROM Students s JOIN Enrollments e
    ON s.StudentID = e.StudentID
    WHERE s.StudentID = @StudentID
    GROUP BY
        s.StudentID,
        s.FirstName,
        s.LastName;
END;
```

StudentID	FirstName	LastName	AverageGrade
1	Ahmed	Ali	89.5

EXEC GetStudentAverageGrade
@StudentID = 1;

```
CREATE PROCEDURE GetDepartmentPerformance
AS
BEGIN
```

```
    SELECT d.DepartmentName, COUNT(s.StudentID) AS
    NumberOfStudents, ROUND(AVG(s.GPA), 2) AS AverageGPA,
    MAX(s.GPA) AS HighestGPA,
    MIN(s.GPA) AS LowestGPA
    FROM Departments d LEFT JOIN Students s
    ON d.DepartmentID = s.DepartmentID
    GROUP BY d.DepartmentName
    ORDER BY AverageGPA DESC;
```

```
END;
```

Performance of each dep:

- Number of students
- Average GPA
- Highest GPA
- Lowest GPA

DepartmentName	NumberOfStudents	AverageGPA	HighestGPA	LowestGPA
Information Systems	3	3.73	3.95	3.45
Business Administration	2	3.65	3.7	3.6
Computer Science	3	3.17	3.5	2.9
Engineering	2	2.98	3.2	2.75

```
EXEC GetDepartmentPerformance;
```

CREATE PROCEDURE GetStudentAcademicReport

@StudentID INT

AS

BEGIN

SELECT s.StudentID, s.FirstName + ' ' + s.LastName AS StudentName,

d.DepartmentName,

c.CourseName, e.Grade, e.EnrollmentDate

FROM Students s JOIN Departments d

ON s.DepartmentID = d.DepartmentID

JOIN Enrollments e

ON s.StudentID = e.StudentID

JOIN Courses c

ON e.CourseID = c.CourseID

WHERE s.StudentID = @StudentID

ORDER BY e.Grade DESC;

END;

Academic Report :

- Student name
- Department
- Course name
- Grade
- Enrollment date

StudentID	StudentName	DepartmentName	CourseName	Grade	EnrollmentDate
2	Sara Mohamed	Information Systems	SQL Fundamentals	95	2026-01-15
2	Sara Mohamed	Information Systems	Data Analysis	92	2026-01-15

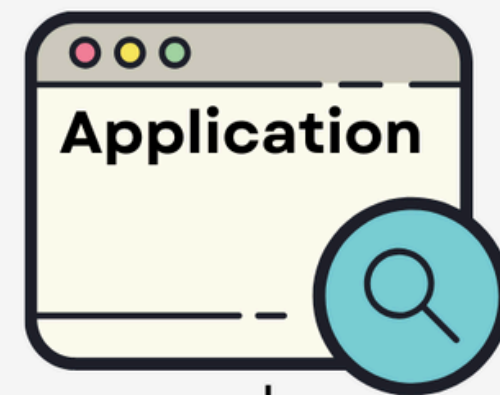
EXEC GetStudentAcademicReport

@StudentID = 2;

Trigger & Event

the specific database action or modification that causes a trigger to execute automatically.

Triggers in



INSERT TRIGGER

UPDATE TRIGGER

DELETE TRIGGER

create new Database

```
CREATE DATABASE Company
```

CREATE TABLE Employees

```
CREATE TABLE Employees  
(  
  EmployeeID INT PRIMARY KEY,  
  EmployeeName VARCHAR(100),  
  Salary float  
)
```

INSERT INTO Employees

```
INSERT INTO Employees  
VALUES
```

```
(1, 'Ahmed', 10000),  
(2, 'Sara', 12000),  
(3, 'Omar', 9000),  
(4, 'Mariam', 15000)
```

CREATE TABLE Salary

```
CREATE TABLE Salary  
(  
    ID INT PRIMARY KEY,  
    EmployeeID INT,  
    OldSalary float,  
    NewSalary float,  
    ChangeDate DATETIME  
)
```

Create Trigger Named trg_SalaryChange

```
CREATE TRIGGER trg_SalaryChange
ON Employees
AFTER UPDATE
AS
BEGIN
INSERT INTO Salary
(EmployeeID, OldSalary, NewSalary, ChangeDate)
SELECT
d.EmployeeID, d.Salary, i.Salary, GETDATE()
FROM deleted d JOIN inserted i
ON d.EmployeeID = i.EmployeeID
WHERE d.Salary <> i.Salary;
END
```

inserted and deleted are special logical tables automatically provided by SQL

Test the Trigger

```
UPDATE Employees  
SET Salary = 11000  
WHERE EmployeeID = 1;
```

```
SELECT *  
FROM Employees;
```

```
SELECT *  
FROM Salary;
```

Project



create new data base
upload your file
analysis the data



show salesdetails table , salesReturn table

```
select *  
from SalesDetails
```

```
select *  
from SalesReturns
```



get only the customer that make orders

```
select Customer  
from SalesDetails inner join Customers  
on Customers.CustomerKey = SalesDetails.CustomerKey
```



get the customers that make returns

```
select Customer  
from SalesReturns inner join Customers  
on Customers.CustomerKey = SalesReturns.CustomerKey
```



get the customers that not make Orders

```
select Customer , SalesDetails.CustomerKey as "id_SD", Customers.CustomerKey  
from Customers left join SalesDetails  
on Customers.CustomerKey = SalesDetails.CustomerKey  
where SalesDetails.CustomerKey is null
```



get the top (3) returned Product

```
select top (3) *  
from Product join SalesReturns  
on Product.ProductKey = SalesReturns.ProductKey  
order by ReturnQuantity desc
```



get the top (5) customers make return

```
select top (5) *  
from Customers join SalesReturns  
on Customers.CustomerKey = SalesReturns.CustomerKey  
order by ReturnQuantity desc
```



unique customer ?

```
select distinct customerkey  
from salesdetails
```



what is the total number of customer ?

```
select count(distinct customerkey)  
from salesdetails
```



what is the total number of customer ?

```
select count(distinct customerkey)  
from customers
```



what is the total revenue ?

```
select sum(totalamount)  
from salesdetails
```



what is the average of qty sold ?

```
select AVG(orderquantity)  
from salesdetails
```



get the biggest product according to QTY

```
select MAX(orderquantity)  
from salesdetails
```



get the smallest product according to QTY

```
select MIN(orderquantity)  
from salesdetails
```



what is the revenue Per continent

```
select continent , sum(totalamount)  
from customers join salesdetails  
on customers.customerkey = salesdetails.customerkey  
group by continent
```



what is the total qty sold by each category

```
select categoryname , sum(totalamount)  
from product join salesdetails  
on product.productkey = salesdetails.productkey  
group by categoryname
```



get the total revenue in each year

```
select year(orderdate) , sum(totalamount)
      from salesdetails
      group by year(orderdate)
```



get the total qty in each month

```
select month(orderdate) , sum(totalamount)
      from salesdetails
      group by month(orderdate)
      order by month(orderdate)
```



**get the orders amount/price that is greater
than the average of total revenue / total amount**

```
select salesdetailskey  
  from salesdetails  
 where totalamount >  
      (  
select AVG(totalamount)  
  from salesdetails  
      )
```



get the product name , its sold qty , its returned qty

```
select productname , orderquantity , ReturnQuantity  
      from product join SalesDetails  
      on Product.ProductKey = SalesDetails.ProductKey  
      join SalesReturns  
      on Product.ProductKey = SalesReturns.ProductKey
```



--use case = if condition

with new_table as

(

select productname , orderquantity , ReturnQuantity

from product join SalesDetails

on Product.ProductKey = SalesDetails.ProductKey

join SalesReturns

on Product.ProductKey = SalesReturns.ProductKey

)

select ProductName , sum(orderquantity) O_Qty , sum(ReturnQuantity) R_Qty,

case

when sum(orderquantity) > sum(ReturnQuantity) then 'Safe'

when sum(orderquantity) < sum(ReturnQuantity) then 'not safe'

when sum(orderquantity) = sum(ReturnQuantity) then 'check'

end status_product

from new_table

group by ProductName



use CTE (Common Table Expression)

```
with new_table as
(
select productname , orderquantity , ReturnQuantity
from product join SalesDetails
on Product.ProductKey = SalesDetails.ProductKey
join SalesReturns
on Product.ProductKey = SalesReturns.ProductKey
)
select ProductName , avg(orderquantity) OR_Q
from new_table
group by ProductName
```